

Optimization, Backpropagation, and Stochastic Gradients

Mathematics of Deep Learning — Chapter 5

Yin Yang

Foundation Books Project

Roadmap

Training as Optimization

Derivatives, Gradients, and Local Linear Models

Gradient Descent and Learning Rates

Computational Graphs and the Chain Rule

Backpropagation Through Layers

Backward Through Convolutional Layers

Softmax and Cross-Entropy Gradients

Minibatches and Stochastic Gradients

Momentum, Adaptive Methods, Decay, Clipping

Numerical Stability and Gradient Pathologies

Reading Training Curves

Automatic Differentiation and the Training Loop

Summary

Training as Optimization

From Empirical Risk to a Numerical Method

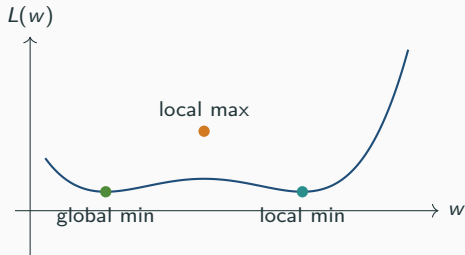
Training chooses parameters by making a scalar objective small:

$$\theta^* \in \arg \min_{\theta} \hat{R}(\theta), \quad \hat{R}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i).$$

- Deep-learning losses are usually *nonconvex* and *high-dimensional*.
- No closed-form θ^* — training produces a sequence $\theta_0, \theta_1, \theta_2, \dots$
- Monitor $\hat{R}(\theta_t)$ and validation metrics, not a global minimum certificate.

“Approximately minimize” is the right verb — the optimizer sees only local derivative information.

Stationary Points and the Nonconvex Landscape



Quartic $L(w) = w^4 - 3w^2 + w$ has multiple turning points.

- Stationary point: $\nabla L(\theta) = 0$ — no first-order direction to follow.
- Local vs. global minima; saddle points are stationary but neither.
- Different initializations can flow to different low-loss regions.

Initialization matters because the optimizer only sees local slope.

Derivatives, Gradients, and Local Linear Models

Derivative as a Local Rate

For a one-parameter loss $L(w)$:

$$\frac{dL}{dw}(w) = \lim_{\epsilon \rightarrow 0} \frac{L(w + \epsilon) - L(w)}{\epsilon}.$$

Example: $L(w) = (w - 3)^2 \Rightarrow dL/dw = 2(w - 3)$.

Point	dL/dw	Local message
$w = 0$	-6	small $+\Delta w$ decreases L
$w = 3$	0	stationary point (the minimum)
$w = 4$	+2	small $+\Delta w$ increases L

The derivative's sign names which nearby direction lowers the loss — nothing more, nothing less.

Gradient: All Partial at Once

For $\theta = (\theta_1, \dots, \theta_m)$:

$$\nabla L(\theta) = \begin{bmatrix} \partial L / \partial \theta_1 \\ \vdots \\ \partial L / \partial \theta_m \end{bmatrix}.$$

Tiny model $\hat{y} = wx + b$, $L = (\hat{y} - y)^2$:

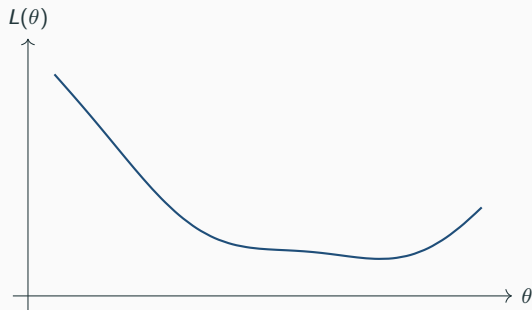
$$\frac{\partial L}{\partial w} = 2x(wx + b - y), \quad \frac{\partial L}{\partial b} = 2(wx + b - y).$$

First-order Taylor approximation:

$$L(\theta + \Delta\theta) \approx L(\theta) + \nabla L(\theta)^\top \Delta\theta.$$

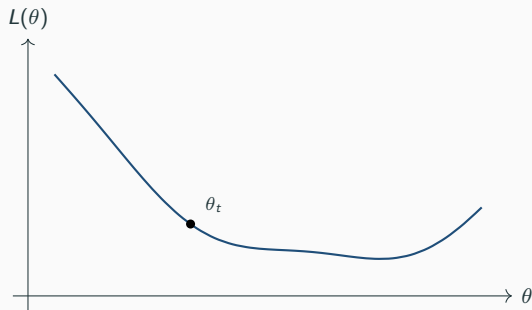
Each partial isolates one parameter's local effect on the same scalar loss.

Local Linear Model, Step by Step



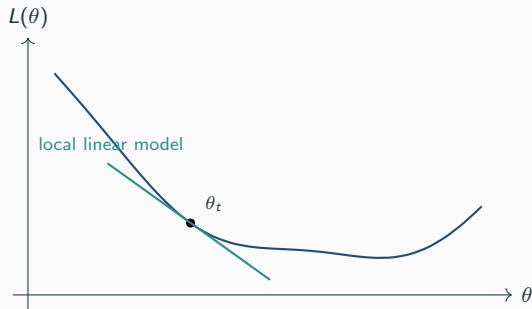
(1) True loss curve is nonconvex.

Local Linear Model, Step by Step



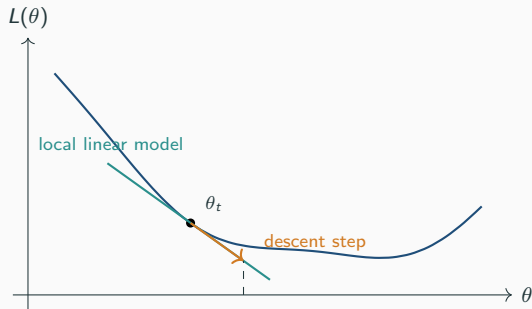
- (1) True loss curve is nonconvex.
- (2) Mark the current parameter θ_t .

Local Linear Model, Step by Step



- (1) True loss curve is nonconvex.
- (2) Mark the current parameter θ_t .
- (3) The gradient gives a local linear model.

Local Linear Model, Step by Step



- (1) True loss curve is nonconvex.
- (2) Mark the current parameter θ_t .
- (3) The gradient gives a local linear model.
- (4) A negative-gradient step follows the model downhill — check the curve afterward.

First-order approximations are reliable only for small moves.

Steepest Descent under the Euclidean Norm

Directional derivative along unit u : $\nabla L(\theta)^\top u$. **Cauchy–Schwarz**: for $\|u\|_2 = 1$,

$$\nabla L(\theta)^\top u \geq -\|\nabla L(\theta)\|_2,$$

with equality when $u = -\nabla L(\theta)/\|\nabla L(\theta)\|_2$. Sanity check on a step in $d = -\nabla L$:

$$\nabla L^\top d = -\|\nabla L\|_2^2 < 0.$$

Example: $\nabla L = (4, -2)$, $\Delta\theta = (-0.1, 0.05)$ gives predicted change
 $4(-0.1) + (-2)(0.05) = -0.5$.

The negative gradient is the steepest first-order descent direction — under the Euclidean geometry.

Gradient Descent and Learning Rates

The Gradient Descent Update

$$\theta_{t+1} = \theta_t - \eta \nabla \hat{R}(\theta_t), \quad \eta > 0.$$

One-parameter quadratic $L(w) = (w - 3)^2$, start $w_0 = 0$:

$$w_{t+1} - 3 = (1 - 2\eta)(w_t - 3) \Rightarrow w_t - 3 = (1 - 2\eta)^t(w_0 - 3).$$

Range of η	Behavior
$0 < \eta < 1/2$	monotone shrinkage
$1/2 < \eta < 1$	oscillating shrinkage
$\eta = 1$	oscillation, no progress
$\eta > 1$	divergence

The same descent *direction* can converge, oscillate, or diverge depending on the step size.

Learning Rate vs. Curvature

Anisotropic example:

$$L(w_1, w_2) = 10w_1^2 + w_2^2.$$

Steep along w_1 , gentle along w_2 . A safe η for w_2 overshoots along w_1 .

Local condition number

For Hessian $H \succ 0$, $\kappa(H) = \lambda_{\max}(H)/\lambda_{\min}(H)$. Large κ means *poor conditioning* — curvature varies dramatically across directions.

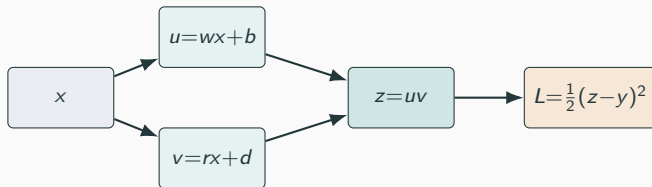
Behavior visible in loss curves before it shows up in accuracy:

- Too small $\eta \Rightarrow$ slow but stable decrease.
- Too large $\eta \Rightarrow$ oscillation, divergence, NaNs.

Momentum and adaptive optimizers exist because one global η rarely fits every coordinate.

Computational Graphs and the Chain Rule

Forward and Backward, Built Up

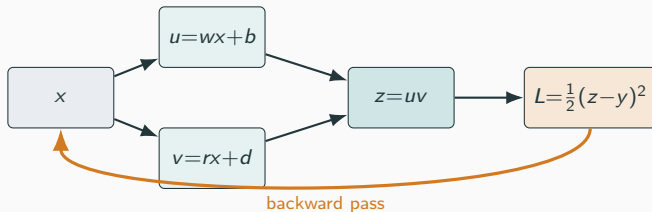


(1) Forward pass: compute u, v, z, L left to right.

Branching x : contributions *add*:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial u} w + \frac{\partial L}{\partial v} r.$$

Forward and Backward, Built Up



- (1) Forward pass: compute u, v, z, L left to right.
- (2) Backward pass: start with $\partial L / \partial L = 1$ and apply the local rule right to left.

Branching x : contributions *add*:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial u} w + \frac{\partial L}{\partial v} r.$$

Backprop reuses each local derivative once instead of recomputing whole paths.

Reverse-Mode Recursion, on Numbers

Let $x = 2$, $w = 1.5$, $b = -1$, $r = -0.25$, $d = 1.5$, $y = 1.25$. **Forward:** $u = 2$, $v = 1$, $z = 2$, $L = \frac{1}{2}(0.75)^2 = 0.28125$. **Backward**, starting from $\partial L / \partial z = z - y = 0.75$:

$$\frac{\partial L}{\partial u} = 0.75 \cdot v = 0.75, \quad \frac{\partial L}{\partial v} = 0.75 \cdot u = 1.5.$$

Then

$$\frac{\partial L}{\partial w} = 0.75 \cdot x = 1.5, \quad \frac{\partial L}{\partial b} = 0.75, \quad \frac{\partial L}{\partial r} = 1.5 \cdot x = 3.$$

Branch sum: $\partial L / \partial x = 0.75(1.5) + 1.5(-0.25) = 0.75$. Each node stores one number during backward: the loss's sensitivity to that node.

Backpropagation Through Layers

Affine Layer Backward Pass

Forward: $z = Wx + b$, with $x \in \mathbb{R}^d$, $W \in \mathbb{R}^{r \times d}$, $b \in \mathbb{R}^r$. Upstream gradient $g_z = \partial L / \partial z \in \mathbb{R}^r$.

Shape-aware backprop

$$\frac{\partial L}{\partial W} = g_z x^\top \in \mathbb{R}^{r \times d}, \quad \frac{\partial L}{\partial b} = g_z \in \mathbb{R}^r, \quad \frac{\partial L}{\partial x} = W^\top g_z \in \mathbb{R}^d.$$

Quantity	Shape	Read as
g_z	r	upstream signal
$\partial L / \partial W$	$r \times d$	for the optimizer
$\partial L / \partial x$	d	passed to previous layer

Every gradient has the same shape as the quantity it differentiates with respect to.

One Affine Layer in Numbers

Let

$$W = \begin{bmatrix} 1 & 2 \\ -1 & 3 \end{bmatrix}, x = \begin{bmatrix} 4 \\ 5 \end{bmatrix}, b = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \Rightarrow z = \begin{bmatrix} 14 \\ 12 \end{bmatrix}.$$

With upstream $g_z = (0.5, -2)^\top$:

$$\frac{\partial L}{\partial W} = g_z x^\top = \begin{bmatrix} 2 & 2.5 \\ -8 & -10 \end{bmatrix}, \quad \frac{\partial L}{\partial x} = W^\top g_z = \begin{bmatrix} 2.5 \\ -5 \end{bmatrix}.$$

Elementwise activation $h = \phi(z)$: $g_z = g_h \odot \phi'(z)$. For ReLU, $\text{ReLU}'(u) = \mathbf{1}\{u > 0\}$.

Minibatch row form $Z = XW^\top + \mathbf{1}b^\top$ gives $\partial L / \partial W = G_Z^\top X$. Shapes are the first debugging tool when a layer fails to train.

Backward Through Convolutional Layers

Convolution: Accumulate and Scatter

Forward (stride 1, valid, channel-last):

$$Y_{n,i,j,q} = b_q + \sum_{u,v,c} K_{u,v,c,q} X_{n,i+u,j+v,c}.$$

Backward pattern

- $\partial L / \partial K$ *accumulates*: a shared weight is reused at every output location, so its gradient is a sum over those uses (correlation-like).
- $\partial L / \partial X$ *scatters*: each upstream G_Y value flows back through the kernel offsets that touched the input pixel (transpose of the linear map).

Padding, stride, dilation only change which (u, v) are valid. A CNN backward pass is ordinary layer backprop — weight sharing just makes gradients accumulate across space.

Tiny CNN Backward Pass

3×3 input, 2×2 kernel, no bias:

$$X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{bmatrix}, K = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix}, G_Y = \begin{bmatrix} 1 & 0.5 \\ -1 & 2 \end{bmatrix}.$$

Kernel weight $k_{0,0}$ is reused at all four output windows:

$$\frac{\partial L}{\partial k_{0,0}} = 1(1) + 0.5(2) + (-1)(0) + 2(1) = 4.$$

Center input $x_{1,1}$ receives four scattered contributions:

$$\frac{\partial L}{\partial x_{1,1}} = 1(2) + 0.5(0) + (-1)(-1) + 2(1) = 5.$$

Full kernel gradient: $\partial L / \partial K = \begin{bmatrix} 4 & 7 \\ 0.5 & 1.5 \end{bmatrix}$. Pooling backward follows the same logic: average-pool spreads, max-pool routes to the argmax.

Softmax and Cross-Entropy Gradients

Logit Gradient: $p - e_y$

Logits $z \in \mathbb{R}^K$, probabilities $p_k = e^{z_k} / \sum_j e^{z_j}$, loss $\ell(z, y) = -\log p_y = -z_y + \log \sum_j e^{z_j}$.

Proposition

$$\nabla_z \ell(z, y) = p - e_y, \quad \frac{\partial \ell}{\partial z_k} = p_k - \mathbf{1}\{k = y\}.$$

Three-class example: $p = (0.10, 0.70, 0.20)$, true class $y = 2$:

$$p - e_y = (0.10, -0.30, 0.20).$$

Correct class gets a negative gradient (raise its logit); competitors get positive gradients (lower theirs). For softmax cross-entropy the logit gradient is exactly “predicted minus target.”

Batched Form and Confident-Wrong Behavior

Predicted probabilities $P \in \mathbb{R}^{m \times K}$, one-hot targets $Y \in \mathbb{R}^{m \times K}$. Average cross-entropy:

$$L = \frac{1}{m} \sum_{i=1}^m -\log p_{i,y_i} \Rightarrow \frac{\partial L}{\partial Z} = \frac{1}{m}(P - Y).$$

- Loss $-\log p_y$ is unbounded when $p_y \rightarrow 0$ (confident wrong).
- Gradient $p - e_y$ is bounded in ℓ_∞ by 1.
- Hence numerically stable softmax/log-sum-exp is essential (next section).

This clean form is why the classifier head trains cleanly even with deep feature extractors below.

Minibatches and Stochastic Gradients

Minibatch Gradient as an Estimator

Full empirical gradient:

$$\nabla \hat{R}(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla_{\theta} \ell(\theta; x_i, y_i).$$

Minibatch B of fixed size b :

$$g_B(\theta) = \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \ell(\theta; x_i, y_i).$$

Unbiasedness

Uniform sampling $\Rightarrow \mathbb{E}[g_B(\theta)] = \nabla \hat{R}(\theta)$.

Example: per-example gradients $2, 4, -1, 3 \Rightarrow$ full mean 2.

$B_1 = \{1, 2\} : g = 3$; $B_2 = \{3, 4\} : g = 1$. **Unbiased means correct on average — not correct on every batch.**

SGD: Noise vs. Progress

Smooth nonconvex result: with $\mathbb{E}[\|g_k - \nabla F(\theta_k)\|^2] \leq \sigma^2$ and $0 < \eta \leq 1/L_s$,

$$\frac{1}{T} \sum_{k=0}^{T-1} \mathbb{E}[\|\nabla F(\theta_k)\|^2] \leq \frac{2(F(\theta_0) - F_\star)}{\eta T} + \eta L_s \sigma^2.$$

Batch size	Gradient	Engineering tradeoff
Very small	Noisy estimate	Cheap; jagged loss curve
Moderate	Balanced	Usual starting point
Very large	Low variance	Memory cost; fewer updates/epoch

The bound does not promise a global minimum — it promises that average gradient norm can be made small.

Momentum, Adaptive Methods, Decay, Clipping

Four Variants on the Same Gradient Step

Momentum

$$\Delta_{t+1} = \mu\Delta_t - \eta g_t$$

Momentum: damp zigzags, build consistent direction.

Four Variants on the Same Gradient Step

Momentum

$$\Delta_{t+1} = \mu\Delta_t - \eta g_t$$

Adam

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

Momentum: damp zigzags, build consistent direction.

Adam: coordinate-wise scaling via running squared gradients.

Four Variants on the Same Gradient Step

Momentum

$$\Delta_{t+1} = \mu\Delta_t - \eta g_t$$

Adam

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

Weight decay (AdamW)

$$(1 - \eta\lambda)\theta_t - \eta \text{step}$$

Momentum: damp zigzags, build consistent direction.

Adam: coordinate-wise scaling via running squared gradients.

Weight decay: shrink parameters separately from the gradient step.

Four Variants on the Same Gradient Step

Momentum

$$\Delta_{t+1} = \mu\Delta_t - \eta g_t$$

Adam

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

Weight decay (AdamW)

$$(1 - \eta\lambda)\theta_t - \eta \text{step}$$

Clipping

$$\tilde{g} = c g / \|g\| \text{ if } \|g\| > c$$

Momentum: damp zigzags, build consistent direction.

Adam: coordinate-wise scaling via running squared gradients.

Weight decay: shrink parameters separately from the gradient step.

Clipping: bound update size when gradients spike.

Each method changes the map from gradient to update — not the loss being differentiated.

One Adam Step on Numbers

$g_1 = (1, 10)^\top$, $m_0 = v_0 = 0$, $\beta_1 = 0.9$, $\beta_2 = 0.99$:

$$m_1 = 0.1 g_1 = (0.1, 1)^\top, \quad v_1 = 0.01 (g_1 \odot g_1) = (0.01, 1)^\top.$$

Bias correction: $\hat{m}_1 = m_1/(1 - 0.9) = (1, 10)$, $\hat{v}_1 = v_1/(1 - 0.99) = (1, 100)$. Update direction (ignoring ϵ):

$$\frac{\hat{m}_1}{\sqrt{\hat{v}_1}} = \left(\frac{1}{1}, \frac{10}{10} \right) = (1, 1).$$

Both coordinates take comparable steps despite a $10\times$ gap in raw gradient magnitudes.

Momentum: $g_0 = 4$, $g_1 = 2$, $\mu = 0.9$, $\eta = 0.1$: $\Delta_1 = -0.4$, $\Delta_2 = -0.56$ (second step is *bigger*). Adaptive rescaling makes coordinate-wise step sizes less brittle.

Numerical Stability and Gradient Pathologies

Vanishing and Exploding Products



(1) Backward pass multiplies through layers.

Saturating sigmoid: $\sigma'(u) = \sigma(u)(1 - \sigma(u)) \rightarrow 0$ when $|u|$ is large. Dead ReLU: $u < 0 \Rightarrow$ zero gradient.

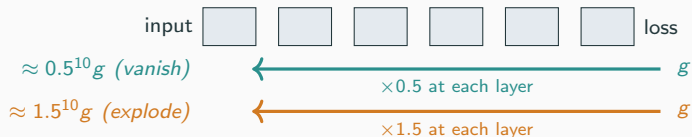
Vanishing and Exploding Products



- (1) Backward pass multiplies through layers.
- (2) Factors below 1 shrink: $0.5^{10} \approx 10^{-3}$.

Saturating sigmoid: $\sigma'(u) = \sigma(u)(1 - \sigma(u)) \rightarrow 0$ when $|u|$ is large. Dead ReLU: $u < 0 \Rightarrow$ zero gradient.

Vanishing and Exploding Products



- (1) Backward pass multiplies through layers.
- (2) Factors below 1 shrink: $0.5^{10} \approx 10^{-3}$.
- (3) Factors above 1 amplify: $1.5^{10} \approx 57.7$.

Saturating sigmoid: $\sigma'(u) = \sigma(u)(1 - \sigma(u)) \rightarrow 0$ when $|u|$ is large. Dead ReLU: $u < 0 \Rightarrow$ zero gradient. Xavier/He initialization aims to keep per-layer signal scale near 1.

Stable Softmax via Log-Sum-Exp

Softmax is shift-invariant:

$$\text{softmax}(z - c\mathbf{1})_k = \text{softmax}(z)_k.$$

Direct computation with $z = (1000, 1001, 998)$ overflows. Use $c = \max_j z_j = 1001$:

$$z - c\mathbf{1} = (-1, 0, -3), \quad \exp(z - c\mathbf{1}) \approx (0.368, 1, 0.050).$$

$$\text{softmax}(z) \approx (0.260, 0.705, 0.035).$$

Symptom	Clue	Response
NaN loss	overflow / log 0	stable log-sum-exp; lower η
Frozen early layers	tiny back-prop product	rescale init / activation
Huge update norms	exploding gradient	clip; check data scale

Most NaN losses are stability bugs, not optimizer failures.

Reading Training Curves

Three Curves, Three Diagnostics

$$L_t^{\text{train}} = \widehat{R}_{\mathcal{D}_{\text{train}}}(\theta_t), \quad L_t^{\text{val}} = \widehat{R}_{\mathcal{D}_{\text{val}}}(\theta_t), \quad G_t = \|\nabla L_{B_t}(\theta_t)\|.$$

Empirical gap along the trajectory: $g_t = L_t^{\text{val}} - L_t^{\text{train}}$.

Pattern	Likely cause	Mathematical clue
Train diverges/oscillates	step too large vs. curvature	descent lemma violated
Train falls, val rises	overfitting along trajectory	g_t grows
Both flat	stalled gradient signal	G_t small early

Read the loss curve before tuning the optimizer — the curve tells you which assumption is failing.

Descent Lemma and Early Stopping

Descent lemma

If ∇L is β -Lipschitz, then for $\theta^+ = \theta - \eta \nabla L(\theta)$,

$$L(\theta^+) \leq L(\theta) - \eta \left(1 - \frac{\eta\beta}{2}\right) \|\nabla L(\theta)\|^2.$$

Decrease is guaranteed for $0 < \eta < 2/\beta$.

Early stopping as trajectory selection

$$\tau \in \arg \min_{0 \leq t \leq T} L_t^{\text{val}}, \quad \hat{\theta} = \theta_\tau.$$

Accuracy reads decision boundaries; cross-entropy reads probabilities — they can move on different schedules.

The optimizer generates a path; validation chooses a point on it.

Automatic Differentiation and the Training Loop

Reverse-Mode AD in One Picture

Straight-line computation $v_j = \phi_j((v_i)_{i \in \text{pa}(j)})$, scalar loss $L = v_m$. Adjoint $\bar{v}_j = \partial L / \partial v_j$ with $\bar{v}_m = 1$.

$$\bar{v}_i \leftarrow \bar{v}_i + \bar{v}_j \frac{\partial v_j}{\partial v_i} \quad (\text{accumulate over outgoing edges}).$$

Baur–Strassen, informally

All first partials of a scalar straight-line computation can be computed with only constant-factor overhead vs. the forward evaluation.

AD differentiates the computation that *actually ran* (including random masks, conditionals, padding). **One backward sweep yields $\nabla_{\theta} L$ for all p parameters — not p separate passes.**

The Whole Training Step, End to End

batch
 B_t

(1) Sample a minibatch.

The Whole Training Step, End to End



- (1) Sample a minibatch.
- (2) Forward pass through composed layers.

The Whole Training Step, End to End



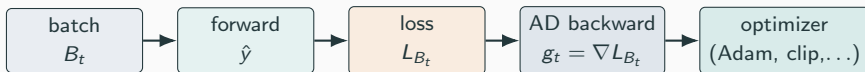
- (1) Sample a minibatch.
- (2) Forward pass through composed layers.
- (3) Scalar loss for that batch.

The Whole Training Step, End to End



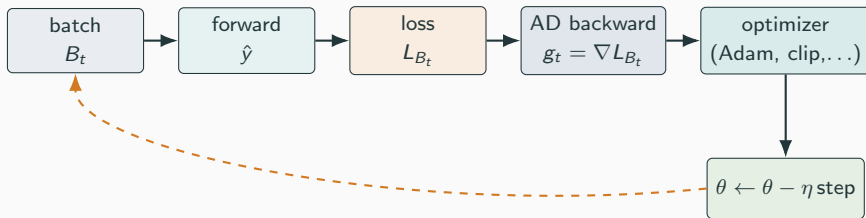
- (1) Sample a minibatch.
- (2) Forward pass through composed layers.
- (3) Scalar loss for that batch.
- (4) Reverse-mode AD returns g_t in one sweep.

The Whole Training Step, End to End



- (1) Sample a minibatch.
- (2) Forward pass through composed layers.
- (3) Scalar loss for that batch.
- (4) Reverse-mode AD returns g_t in one sweep.
- (5) Optimizer maps g_t to a parameter step.

The Whole Training Step, End to End



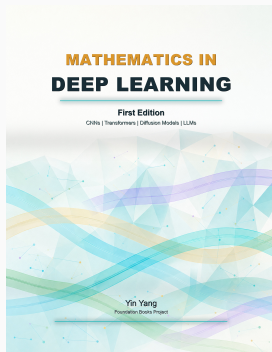
- (1) Sample a minibatch.
- (2) Forward pass through composed layers.
- (3) Scalar loss for that batch.
- (4) Reverse-mode AD returns g_t in one sweep.
- (5) Optimizer maps g_t to a parameter step.
- (6) Update parameters and loop.

Every loop in deep learning is a variation on this six-step structure.

Summary

- **Goal:** minimize $\hat{R}(\theta)$ by following local derivative information.
- **Gradient descent:** same direction, very different behavior depending on η vs. curvature.
- **Backprop = chain rule with bookkeeping:** affine layers give $g_W x^\top$, $W^\top g_z$; convolutions *accumulate* and *scatter*.
- **Softmax cross-entropy:** logit gradient is $p - e_y$.
- **SGD:** minibatch gradient is unbiased; noise vs. cost tradeoff.
- **Variants:** momentum, Adam, weight decay, clipping — all reshape the gradient-to-step map.
- **Stability:** log-sum-exp, variance-preserving init, vanishing/exploding products.
- **Curves:** divergence, overfitting, stalling each have a mathematical signature.
- **AD:** one reverse sweep computes all parameter gradients of the scalar loss that actually ran.

Next: regularization, generalization theory, and the architectural choices that shape what gradients see.



Mathematics in Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	foundation-books.github.io/mathematics-in-deep-learning
Code	github.com/foundation-books/mathematics-in-deep-learning-code
Paperback	amazon.com/dp/BOH1ZZHDT2
Hardcover	amazon.com/dp/BOH1ZL11MJ
Kindle	amazon.com/dp/B0GX32M8SP
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.